

第七章 对象群体的组织

目录

- 7.1 对象数组
- 7.3 集合 (**Collection, Map**)
- 7.4 本章小结

7.1 对象数组

- 数组

- 在Java提供的存储及随机访问对象序列的各种方法中，数组是效率最高的一种
 - 类型检查
 - 边界检查
- 优点
 - 数组知道其元素的类型
 - 编译时的类型检查
 - 大小已知
- 代价
 - 数组对象的大小是固定的，在生存期内大小不可变

7.1 对象数组(续)

- 对象数组
 - 数组元素是类的对象
 - 所有元素具有相同的类型
 - 每个元素都是一个对象的引用

对象数组

7.1.1 用数组存储对象(续)

——对象数组的初始化

对象数组

- 静态初始化：在声明和定义数组的同时对数组元素进行初始化，例如：
 - `BankAccount[] accounts = {
 new BankAccount("Zhang", 100.00),
 new BankAccount("Li", 2380.00),
 new BankAccount("Wang", 500.00),
 new BankAccount("Liu", 175.56),
 new BankAccount("Ma", 924.02)};`
- 动态初始化：使用运算符**new**，需要经过两步：
 - 首先给数组分配空间
`type arrayName[ ]=new type[arraySize];`
 - 然后给每一个数组元素分配空间
`arrayName[0]=new type(paramList);`
...
`arrayName[arraySize-1]=new type(paramList);`

7.1.1 用数组存储对象(续)

——例7_1

对象数组

- 使用数组存储一个班的学生信息及考试成绩。学生信息包括学号、姓名、三门课（英语、数学、计算机）的成绩及总成绩。
- 首先声明学生类**Student**
 - 属性包括
 - 学号（id），姓名（name），英语成绩（eng），数学成绩（math），计算机成绩（comp），总成绩（sum）
 - 方法包括
 - 构造方法，get方法，set方法，toString方法，equals方法，compare方法（比较两个学生的总成绩，结果分大于，小于，等于），sum方法（计算总成绩）
 - 实现**Serializable**接口，以便对象持久化，

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
import java.io.*;
public class Student implements Serializable {
    private String id;           //学号
    private String name;         //姓名
    private int eng;             //英语成绩
    private int math;            //数学成绩
    private int comp;            //计算机成绩
    private int sum;             //总成绩
    public Student(String id,String name,int eng,int math,int comp){
        this.id=id;
        this.name=name;
        this.eng=eng;
        this.math=math;
        this.comp=comp;
        sum();                   //计算总成绩
    }
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象
数组

```
public Student(Student s){
    this.id=s.id;
    this.name=new String(s.name);
    this.eng=s.eng;
    this.math=s.math;
    this.comp=s.comp;
    sum();          //计算总成绩
}

public void setId(String id){
    this.id=id;
}

public void setName(String name){
    this.name=name;
}
```


7.1.1 用数组存储对象(续)

——例7_1

对象
数组

```
public void setEng(int eng){
    this.eng=eng;
    sum();        //计算总成绩
}
public void setMath(int math){
    this.math=math;
    sum();        //计算总成绩
}
public void setComp(int comp){
    this.comp=comp;
    sum();        //计算总成绩
}
public String getId(){
    return id;
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象
数组

```
public String getName(){
    return name;
}
public int getEng(){
    return eng;
}
public int getMath(){
    return math;
}
public int getComp(){
    return comp;
}
public int getSum(){
    return sum;
}
void sum(){
    this.sum=eng+math+comp;
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
public String toString(){
    return getId() + "\t"+getName() + "\t"+getEng() +
        "\t"+getMath() + "\t"+getComp() + "\t"+getSum();
}
public boolean equals(Object x) {
    if (this.getClass() != x.getClass()) return false;
    Student b = (Student) x;
    return (this.getId().equals(b.getId()));
}
//比较成绩大小,当前对象成绩比参数对象成绩大时返回1,相等时返回0,其它返回-1.
public int compare(Student A){
    if(this.getSum()>A.getSum()) return 1;
    else if(this.getSum()==A.getSum()) return 0;
    else return -1;
}
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象数组

- 下面声明班级类StudentClass:

- 属性包括

- 班级名称 (name)，容量 (capacity)，学生 (students)，实际人数 (size)。

- 方法包括

- 构造方法，get方法，set方法，toString方法。

```
public class StudentClass{  
    private String name;           //班级名称  
    static int capacity = 40;      //最大容量  
    private Student students[];    //学生  
    private int size;              //实际人数
```

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
public StudentClass(String name, int size){
    this.name = name;
    this.size = size;
    students = new Student[capacity];
}
public String getName(){
    return name;
}
public int getCapacity(){
    return capacity;
}
public Student[] getStudents(){
    return students;
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象
数组

```
public int getSize(){
    return size;
}
public void setName(String name){
    this.name = name;
}
public void setCapacity(int capacity){
    this.capacity = capacity;
}
public void setSize(int size){
    this.size = size;
}
public void setStudents(Student[] students){
    for (int i = 0; i<size; i++)
        this.students[i] = new Student(students[i]);
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
public String toString(){
    String s;
    s = " 班级:" + name + "\t" + "容量:" + capacity + "\t" +
        "实际人数:" + size + "\n\n";
    s = s + "学号" + "\t" + "姓名" + "\t" + "英语" + "\t" + "数
学" + "\t" + "计算机" + "\t" + "总成绩\n";
    for (int i=0; i<size; i++)
        s = s + students[i].getId()+"\t"+students[i].getName()+"\t"
            +students[i].getEng()+"\t"+students[i].getMath()+"\t"
            +students[i].getComp()+"\t"+students[i].getSum()+"\n";
    return s;
}
}
```

7.1.1 用数组存储对象(续)

——例7_1

- 下面声明测试类**Tester1**（其中**Keyboard**类用于从键盘读数），为测试简单，仅生成具有**5**名学生的班级，**5**名学生的信息从键盘输入，为了避免以后再重复输入，可将输入的学生信息保存到文件中

```
import java.io.*;
public class Tester1{
    public static void main(String args[]){
        Student students[];
        StudentClass aClass = new StudentClass("软件01", 5);
        students=new Student[5];
        for (int i=0; i<5; i++)
            students[i] = new Student(getAStudent(i+1));
        aClass.setStudents(students);
        System.out.println(aClass);
    }
}
```

对象数组

7.1.1 用数组存储对象(续)

——例7_1

//将学生信息保存到文件stu.ser中。

```
try {  
    FileOutputStream fo = new FileOutputStream("stu.ser");  
    ObjectOutputStream so = new ObjectOutputStream(fo);  
    for (int i=0; i<5; i++)  
        so.writeObject(students[i]);  
    so.close();  
}  
catch(Exception e){  
    System.out.println(e) ;  
}  
}
```

对象
数组

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
public static Student getAStudent(int i){  
    Student studenti;  
    System.out.println("输入第" + i + "个学生的信息:");  
    System.out.print("学号:");  
    String id = Keyboard.getString();  
    System.out.print("姓名:");  
    String name = Keyboard.getString();  
    System.out.print("英语成绩:");  
    int eng = Keyboard.getInteger();  
    System.out.print("数学成绩:");  
    int math = Keyboard.getInteger();  
    System.out.print("计算机成绩:");  
    int comp = Keyboard.getInteger();  
    studenti = new Student(id,name,eng,math,comp);  
    return studenti;  
}  
}
```

7.1.1 用数组存储对象(续)

——例7_1

对象数组

```
public class Keyboard {  
    static BufferedReader inputStream = new BufferedReader(new  
        InputStreamReader(System.in));  
  
    public static int getInteger() {  
        try {  
            return (Integer.valueOf(inputStream.readLine().trim()).intValue());  
        } catch (Exception e) {  
            e.printStackTrace();  
            return 0;  
        }  
    }  
  
    public static String getString() {  
        try {  
            return (inputStream.readLine());  
        } catch (IOException e) {  
            return "0";  
        }  
    }  
}
```

7.1.1 用数组存储对象(续)

——例7_1运行结果

- 运行结果如下（其中学生信息的输入只显示一部分）：

输入第1个学生的信息:

学号:250201

姓名:李红

英语成绩:88

数学成绩:76

计算机成绩:60

输入第2个学生的信息:

.....

班级:软件0201 容量:40 实际人数:5

学号	姓名	英语	数学	计算机	总成绩
250201	李红	88	76	60	224
250202	张林	78	67	80	225
250203	董玉梅	86	80	75	241
250204	张力	70	68	75	213
250205	何为	80	90	78	248

7.1.1 用数组存储对象(续)

——为班级类添加查找方法

- 查找

- 也称为检索，就是从一组数据中找出所需的具有某种特征的数据项

- 顺序查找

- 对所存储的数据从第一项开始（也可以从最后一项开始），依次与所要查找的数据进行比较，直到找到该数据或将全部元素都找完还没有找到该数据为止

7.1.1 用数组存储对象(续)

——为班级类添加查找方法

对象数组

- 已知学生的学号，查找此学生是否存在。如果存在，返回其在数组中的下标位置；如果不存在，返回-1。顺序查找方法的代码如下

```
public int find(String id){  
    for (int i = 0; i < size; i++)  
        if (students[i].getId().equals(id)) return i;  
    return -1;  
}
```

7.1.1 用数组存储对象(续)

——为班级类添加增加方法

- 在数组的末尾增加一个学生对象
 - 增加之前需先判断数组中是否还有空间，并且在数组中查找将要增加的学号是否已经存在
 - 增加成功，返回true；否则，返回false

```
public boolean add(Student aStudent){  
    if (size == capacity) return false;  
    if (find(aStudent.getId())>=0) return false;  
    this.students[size] = new Student(  
        new String(aStudent.getId()),  
        new String(aStudent.getName()),  
        aStudent.getEng(),  
        aStudent.getMath(),  
        aStudent.getComp()  
    );  
  
    size++;  
    return true;  
}
```

对象数组

7.1.1 用数组存储对象(续)

——为班级类编写删除方法

- 已知一个**Student**对象，将此对象从数组中删除

```
public boolean del(Student aStudent){  
    int pos = find(aStudent.getId());  
    if (pos == -1) return false;  
    for (int i = pos+1; i<size; i++)  
        students[i-1] = students[i];  
    size--;  
    return true;  
}
```

对象数组

7.1.1 用数组存储对象(续)

——为班级类添加删除方法

- 已知学号,删除一个学生

```
public boolean del(String id){  
    int pos = find(id);  
    if (pos == -1) return false;  
    for (int i = pos+1; i<size; i++)  
        students[i-1] = students[i];  
    size--;  
    return true;  
}
```

7.1.2 对数组元素进行排序

- 排序
 - 按照预先规定的准则（如升序或降序等），把数据有次序地排列起来
 - 已经设计出许多排序算法，常用的排序算法有
 - 选择排序
 - 插入排序
 - 交换排序
 - 以降序为例进行介绍

7.1.2 对数组元素进行排序(续)

——选择排序

- 选择排序的基本思想
 - 先在未排序序列中选一个最小元素，作为已排序子序列
 - 然后再重复地从未排序子序列中选取一个最小元素，把它加到已经排序的序列中，作为已排序子序列的最后一个元素
 - 直到把未排序子序列中的元素处理完为止

7.1.2 对数组元素进行排序(续)

——例7_2

- 用选择排序方法将例7-1中生成的文件 **stu.ser** 中的班级学生按总成绩从高到低排序
 - 在例7-1中的 **StudentClass** 类中增加选择排序方法 **selectionSort**, 代码如下

```
public void selectionSort(){
    Student temp;
    for (int i=0; i<size-1; i++)
        for (int j=i+1; j<size; j++)
            if (students[j].compare(students[i])>0) {
                temp = students[i];
                students[i] = students[j];
                students[j] = temp;
            }
}
```

7.1.2 对数组元素进行排序(续)

——例7_2

测试类代码如下

```
import java.io.*;
public class SortTester{
    public static void main(String args[]){
        Student students[]=new Student[5];
        //从文件stu.ser中读出学生信息
        try {
            FileInputStream fi = new FileInputStream("stu.ser");
            ObjectInputStream si = new ObjectInputStream(fi);
            for (int i=0; i<5; i++)
                students[i] = (Student)si.readObject();
            si.close();
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

对象数组

7.1.2 对数组元素进行排序(续)

——例7_2

对象数组

```
StudentClass aClass = new StudentClass("软件0201",5);
aClass.setStudents(students);
System.out.println(aClass);
// 选择排序
aClass.selectionSort();
System.out.println("选择排序后的结果:\n");
System.out.println(aClass);
}
}
```

7.1.2 对数组元素进行排序(续)

——例7_2运行结果

对象数组

- 运行结果

班级:软件0201 容量:40 实际人数:5

学号	姓名	英语	数学	计算机	总成绩
250201	李红	88	76	60	224
250202	张林	78	67	80	225
250203	董玉梅	86	80	75	241
250204	张力	70	68	75	213
250205	何为	80	90	78	248

选择排序后的结果:

班级:软件0201 容量:40 实际人数:5

学号	姓名	英语	数学	计算机	总成绩
250205	何为	80	90	78	248
250203	董玉梅	86	80	75	241
250202	张林	78	67	80	225
250201	李红	88	76	60	224
250204	张力	70	68	75	213

7.3 集合

- 数组的优点
 - 是Java提供的随机访问对象序列的最有效方法
 - 是一个简单的线性序列，访问元素的速度较快
- 数组的缺点
 - 大小自创建以后就固定了，在其整个生存期内其大小不可改变
 - 数组元素只能是同一类型
- 集合
 - 可动态改变其大小
 - 可在序列中存储不同类型的数据

7.3 集合(续)

- 集合

集合

- 把具有相同性质的一类东西，汇聚成一个整体
- 在Java中有很多与集合有关的接口及类
- 它们被组织在以Collection及Map接口为根的层次结构中，称为集合框架
- 之前，在Java中，没有完整的集合框架。只有一些简单的可以自扩展的容器类
 - Vector
 - Hashtable

7.3.1 Java集合框架

- 集合框架(**Java Collections Framework**)

- 为表示和操作集合而规定的一种统一的标准体系结构
- 提供了一些现成的数据结构可供使用，程序员可以利用集合框架快速编写代码，并获得优良性能
- 包含三大块内容
 - 对外的接口：表示集合的抽象数据类型，使集合的操作与表示分开
 - 接口的实现：指实现集合接口的Java类，是可重用的数据结构
 - 对集合运算的算法：是指执行运算的方法，例如在集合上进行查找和排序

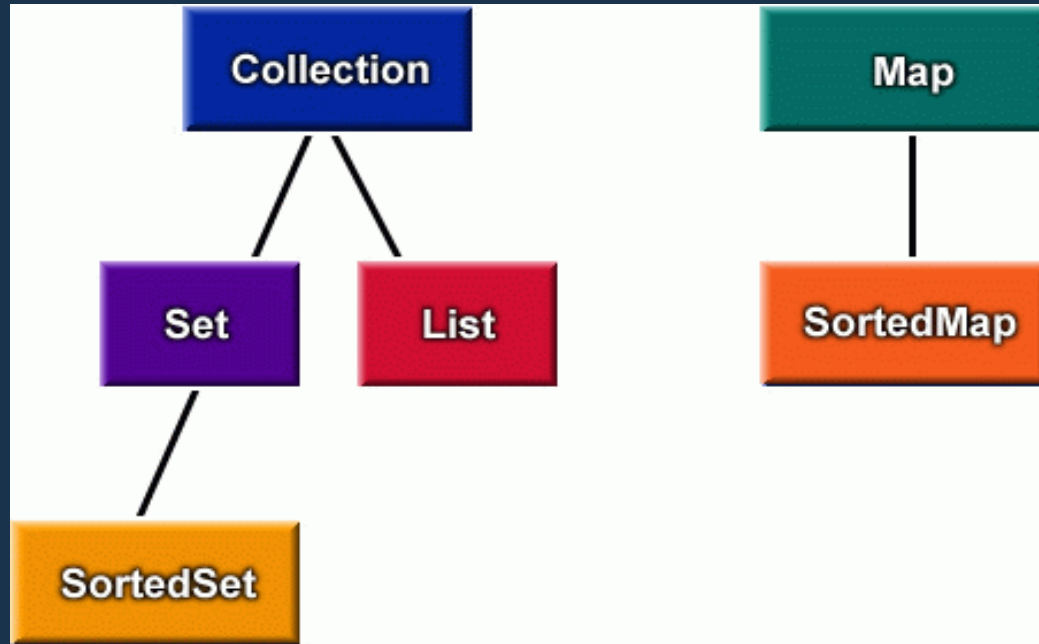
集
合

7.3.1 Java集合框架(续)

——对外的接口

- 集合框架接口
 - 声明了对各种集合类型执行的一般操作
 - 包括Collection、Set、List、SortedSet、Map、SortedMap
 - 基本结构如图

集
合

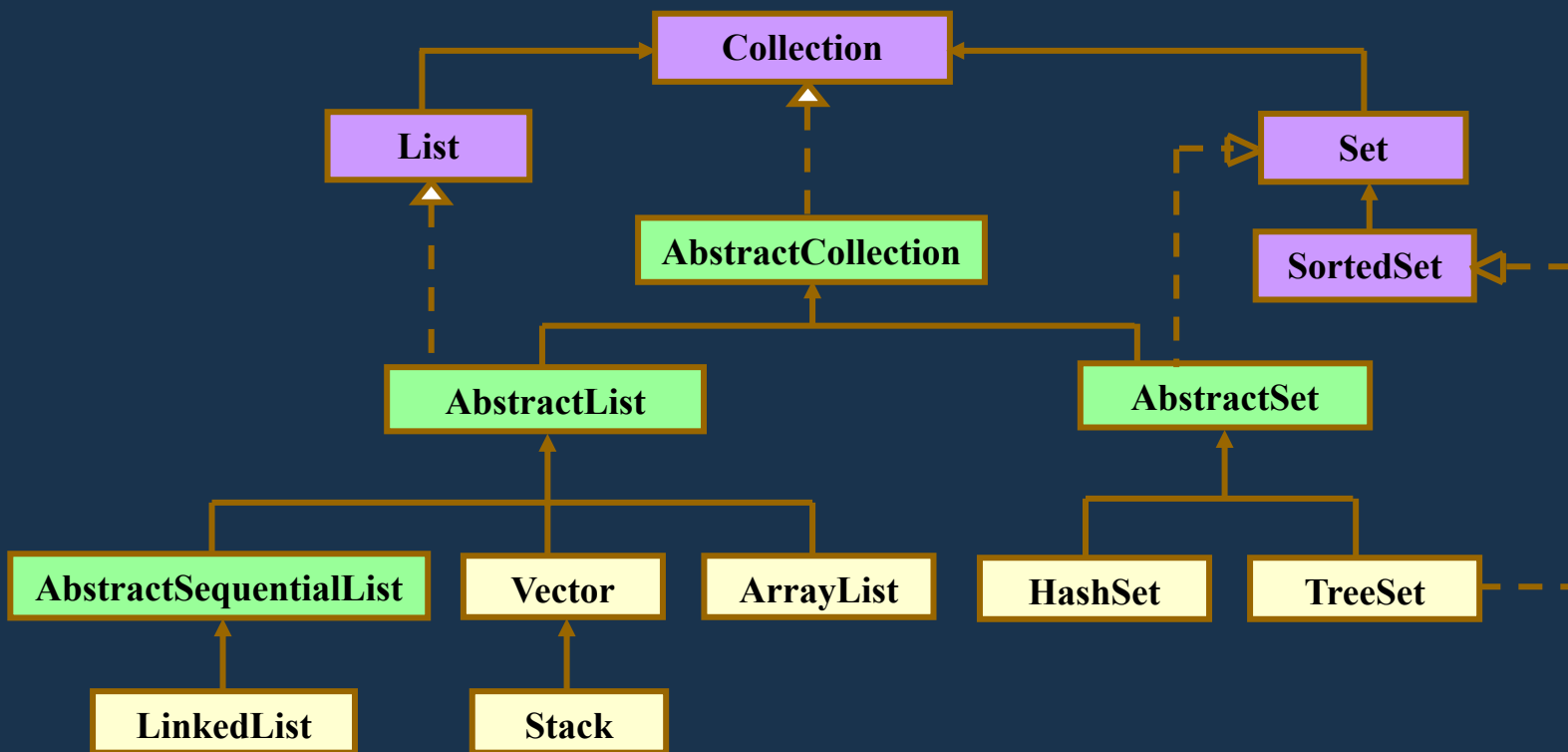


7.3.1 Java集合框架(续)

——Collection接口

- 类层次如图，包括4个接口、4个抽象类及6个具体类

集合



7.3.1 Java集合框架(续)

——Collection接口

- **Collection接口**

- 声明了一组操作成批对象的抽象方法：查询方法、修改方法
- 查询方法
 - `int size()` – 返回集合对象中包含的元素个数
 - `boolean isEmpty()` – 判断集合对象中是否还包含元素，如果没有任何元素，则返回`true`
 - `boolean contains(Object obj)` – 判断对象是否在集合中
 - `boolean containsAll(Collection c)` – 判断方法的接收者对象是否包含集合中的所有元素

集
合

7.3.1 Java集合框架(续)

——Collection接口

集合

— 修改方法包括

- `boolean add(Object obj)` – 向集合中增加对象
- `boolean addAll(Collection c)` – 将参数集合中的所有元素增加到接收者集合中
- `boolean remove(Object obj)` – 从集合中删除对象
- `boolean removeAll(Collection c)` – 将参数集合中的所有元素从接收者集合中删除
- `boolean retainAll(Collection c)` – 在接收者集合中保留参数集合中的所有元素，其它元素都删除
- `void clear()` – 删除集合中的所有元素

7.3.1 Java集合框架(续)

——Set、SortedSet接口

集合

- **Set接口**

- 扩展了Collection
- 禁止重复的元素，是数学中“集合”的抽象
- 对equals和hashCode操作有了更强的约定，如果两个Set对象包含同样的元素，二者便是相等的
- 实现它的两个主要类是哈希集合(HashSet)及树集合(TreeSet)

- **SortedSet接口**

- 一种特殊的Set
- 其中的元素是升序排列的，还增加了与次序相关的操作
- 通常用于存放词汇表这样的内容

7.3.1 Java集合框架(续)

——List接口

- List接口

- 扩展了Collection
- 可包含重复元素
- 元素是有顺序的，每个元素都有一个index值（从0开始）标明元素在列表中的位置
- 实现它的四个主要类是
 - Vector
 - ArrayList: 一种类似数组的形式进行存储，因此它的随机访问速度极快
 - LinkedList: 内部实现是链表，适合于在链表中间需要频繁进行插入和删除操作
 - 栈Stack

集
合

7.3.1 Java集合框架(续)

——Map、SortedMap接口

集合

- **Map接口**

- 不是Collection接口的继承
- 用于维护键/值对(key/value pairs)
- 描述了从不重复的键到值的映射，是一个从关键字到值的映射对象
- 其中不能有重复的关键字，每个关键字最多能够映射到一个值

- **SortedMap接口**

- 一种特殊的Map，其中的关键字是升序排列的
- 与SortedSet对等的Map，通常用于词典和电话目录等

7.3.1 Java集合框架(续)

——接口的实现

集合

- 接口的实现
 - Collection没有直接的实现，只是作为其他集合接口的最小公分母
 - 除Collection 以外，其余五个接口都有实现
 - 主要的实现有
 - Set→HashSet
 - SortedSet→TreeSet
 - List→Vector / ArrayList / LinkedList
 - Map→HashMap
 - SortedMap→TreeMap

7.3.1 Java集合框架(续)

——对集合运算的算法

集合

- 对集合运算的算法
 - 大多数算法都是用于操作List对象
 - 有两个(min和max)可用于任意集合对象
- 排序算法sort
 - 对List重新排序，使其中的元素按照某种次序关系升序排列
 - 有两种形式
 - 简单形式只是将元素按照自然次序排列
 - 第二种形式需要一个附加的Comparator对象作为参数，用于规定比较规则，可用于实现反序或特殊次序排序

7.3.1 Java集合框架(续)

——对集合运算的算法

集合

- 洗牌算法shuffle
 - 其作用与排序算法恰好相反，它打乱List中的任何次序。也就是说以随机方式重排元素，任何次序出现的几率都是相等的
 - 在实现偶然性游戏的时候，这个算法很有用，例如洗牌
- 常规数据处理算法
 - reverse:: 将一个List中的元素反向排列
 - fill: 用指定的值覆写List中的每一个元素，这个操作在重新初始化List时有用
 - copy:: 接受两个参数，目标List和源List，将源中的元素复制到目标，覆写其中的内容。目标List必须至少与源一样长，如果更长，则多余的部分内容不受影响

7.3.1 Java集合框架(续)

——对集合运算的算法

集合

- 查找算法**binarySearch**
 - 使用二分法在一个有序的**List**中查找指定元素
 - 有两种形式
 - 第一种形式假定**List**是按照自然顺序升序排列的
 - 第二种形式需要增加一个**Comparator**对象，表示比较规则，并假定**List**是按照这种规则排序的。
- 寻找最值——用于任何集合对象
 - **min**和**max**算法返回指定集合中的最小值和最大值
 - 这两个算法分别都有两种形式
 - 简单形式按照元素的自然顺序返回最值
 - 另一种形式需要附加一个**Comparator**对象作为参数，并按照**Comparator**对象指定的比较规则返回最值

7.3.1 Java集合框架(续)

——数组实用方法

集合

- Arrays类
 - Java集合框架提供了一套专门用于操作数组的实用方法，它们作为静态方法存在该类中
 - 还包括可以将数组视为列表（List）的静态工厂
 - 常用方法
 - fill (type[] a, type val): 给数组填充，就是简单地把一个数组全部或者某段数据填成一个特殊的值
 - equals (type[] a, type[] b): 实现两个数组的比较，相等时返回true
 - sort (type[] a): 对数组排序
 - binarySearch (): 对数组元素进行二分法查找
 - Arrays.asList (Object[] a): 实现数组到ArrayList的转换

7.3.1 Java集合框架(续)

——数组实用方法

- 数组的填充和复制

```
import java.util.*;
public class CopyingArrays {
    public static void main(String[] args) {
        int[] i = new int[25];
        int[] j = new int[25];
        Arrays.fill(i, 47);
        Arrays.fill(j, 99);
        System.arraycopy(i, 0, j, 0, i.length);
        int[] k = new int[10];
        Arrays.fill(k, 103);
        System.arraycopy(i, 0, k, 0, k.length);
        Arrays.fill(k, 103);
        System.arraycopy(k, 0, i, 0, k.length);
        Integer[] u = new Integer[10];
        Integer[] v = new Integer[5];
        Arrays.fill(u, new Integer(47));
        Arrays.fill(v, new Integer(99));
        System.arraycopy(v, 0, u, u.length/2, v.length);
    }
}
```

集
合

可使用Arrays.toString函数方便的显示出数组的内容

7.3.1 Java集合框架(续)

——数组实用方法

- 数组的比较

```
import java.util.*;
public class ComparingArrays{
    public static void main(String[] args) {
        int[] a1 = new int[10];
        int[] a2 = new int[10];
        Arrays.fill(a1, 47);
        Arrays.fill(a2, 47);
        System.out.println(Arrays.equals(a1, a2)); //true
        a2[3] = 11;
        System.out.println(Arrays.equals(a1, a2)); //false
        String[] s1 = new String[5];
        Arrays.fill(s1, "Hi");
        String[] s2 = {"Hi", "Hi", "Hi", "Hi", "Hi"};
        System.out.println(Arrays.equals(s1, s2)); //true
    }
}
```

集
合

7.3.2 向量(Vector), 动态数组(ArrayList)

- **Vector/ArrayList**

- 实现了Collection接口的具体类
- 能够存储任意对象，但通常情况下，这些不同类型的对象都具有相同的父类或接口
- 不能存储基本类型（**primitive**）的数据，除非将这些数据包裹在包裹类中
- 其容量能够根据空间需要自动扩充
- 增加元素方法的效率较高，除非空间已满，在这种情况下，在增加之前需要先扩充容量

- **Vector**方法是同步的，线程安全

- **ArrayList**方法是非同步的，效率较高

集合

7.3.2 向量(Vector), ArrayList ——构造方法

集合

- **Vector**类的构造方法

- `Vector myVector = new Vector();` //初始容量为10
- `Vector myVector = new Vector(int cap);`
- `Vector myVector = new Vector(Collection col);`
 - 以参数col中的元素进行初始化
 - 也可用数组元素生成, 但需先将数组转换成List对象, 如

```
String[] num = {"one", "two", "three", "four", "five"};  
Vector aVector = new Vector(java.util.Arrays.asList(num));
```

- **ArrayList**的构造方法与**Vector**类似

- `ArrayList myList = new ArrayList();`
- `ArrayList myList = new ArrayList(int cap);`
- `ArrayList myList = new ArrayList(Collection col);`

7.3.2 向量(Vector), ArrayList ——常用方法1

- 本节所有常用方法如无特殊说明，为**Vector**，**ArrayList**类共有之方法
- **void add(Object obj)** ——添加一个对象，如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");
```
- **boolean addAll(Collection col)** ——添加整个集合，如果接收者对象的结果有变化，则返回**true**，如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
Vector yourList = new Vector();  
yourList.addAll(teamList);
```

集
合

7.3.2 向量(Vector), ArrayList ——常用方法2

- **int size()** ——返回元素的个数。
- **boolean isEmpty()** ——如果不含元素，则返回true
- **Object get(int pos)** ——返回指定位置的元素，如

`Vector teamList = new Vector();`

`teamList.add("Zhang Wei");`

`teamList.add("Li Hong");`

`teamList.add("Yu Hongshu");`

`teamList.get(1);` // 返回 "Li Hong"

`teamList.get(3);` // 产生例外 `ArrayIndexOutOfBoundsException`

集
合

7.3.2 向量(Vector), ArrayList ——常用方法3

- **void set(int pos, Object obj)** ——用参数对象替换指定位置的对象，如

```
Vector teamList = new Vector();
```

```
teamList.add("Zhang Wei");
```

```
teamList.add("Li Hong");
```

```
teamList.add("Yu Hongshu");
```

```
teamList.set(2, "Liu Na");
```

```
System.out.println(teamList);
```

```
    // 显示[Zhang Wei, Li Hong, Liu Na]
```

```
teamList.set(3, "Ma Li");
```

```
    // 产生例外ArrayIndexOutOfBoundsException
```

集
合

7.3.2 向量(Vector), ArrayList ——常用方法4

- **boolean remove(Object obj)** ——去除给定对象的第一次出现，如果找到了对象，则返回**true**。去除一个对象后，其后面的所有对象都依次向前移动。

如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
teamList.add("Yu Hongshu");  
teamList.remove("Li Hong");  
teamList.remove("Wang Hong");//不做什么事，也不出现错误  
System.out.println(teamList); // 显示[Zhang Wei,Yu Hongshu]
```

集
合

7.3.2 向量(Vector), ArrayList ——常用方法5

- **Object remove(int pos)** —— 去除给定位置的元素，并返回被去除的对象。如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
teamList.add("Yu Hongshu");  
teamList.remove(0);           //去除Zhang Wei  
teamList.remove(0);           //去除 Li Hong  
System.out.println(teamList); // 显示[Yu Hongshu]  
teamList.remove(1);           //产生例外 ArrayIndexOutOfBoundsException
```

集
合

7.3.2 向量(Vector), ArrayList ——常用方法6

- **boolean removeAll(Collection col)** ——从接收者对象中去除所有在参数对象中出现的元素，如果接收者对象的结果有变化，则返回**true**。如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
teamList.add("Yu Hongshu");  
Vector yourList = new Vector();  
yourList.add("Yu Hongshu");  
yourList.add("He Li");  
yourList.add("Zhang Wei");  
teamList.removeAll(yourList);  
System.out.println(teamList); // 显示[Li Hong]
```

集
合

7.3.2 向量(Vector), ArrayList ——常用方法7

集合

- **void clear()** ——去除所有的元素
- **boolean contains(Object obj)** ——返回是否包含指定的对象，如果包含则返回**true**；否则，返回**false**
- **boolean containsAll(Collection col)** —— 返回是否包含参数**col**中的所有对象
- **int indexOf(Object obj)** ——返回给定对象在**Vector**/**ArrayList**中第一次出现的位置，如不存在，则返回**-1**。如

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
teamList.indexOf("Li Hong");    // 返回1。  
teamList.indexOf("Zhang Li");   // 返回-1。
```

7.3.2 向量(Vector), ArrayList ——常用方法8

- **Enumeration elements()** —— 返回包含Vector中所有元素的Enumeration类对象。该方法只能应用于Vector对象，不能应用于ArrayList对象。如

集
合

```
Vector teamList = new Vector();  
teamList.add("Zhang Wei");  
teamList.add("Li Hong");  
teamList.add("Yu Hongshu");  
teamList.elements();    // 返回Enumeration类对象。
```

- **Iterator iterator()** —— 返回包含Vector/ArrayList中所有元素的Iterator类对象

7.3.2 向量(Vector), ArrayList

——使用Vector存取对象

- 当使用`get()`方法取出Vector中的元素时，返回的类型都是Object类型。Vector的使用者需要记住存入对象的具体类型，当使用`get()`方法取出后，再塑型成其本来的类型。例如，创建Customer类对象的Vector如下：

```
String[] names = {"Zhang", "Li", "Wang", "Zhao"};
Vector v = new Vector();
for (int i=0; i<names.length; i++) {
    Customer c = new Customer();
    c.setName(names[i]);
    v.add(c);
}
```

- 使用`get()`方法将Customer对象从Vector中取出后，需要再塑型成Customer类。代码如下：

```
for (int i=0; i<v.size();i++) {
    Customer c = (Customer) v.get(i) ;
    System.out.println(c.getName());
}
```

7.3.2 向量(Vector), ArrayList

——使用Vector存取数据

- 与所有的集合类一样，**Vector**不能存储原始类型（**primitive**）的数据，如果要存储，则需要使用包裹类。例如，

```
Vector<Double> rateVector = new Vector <Double>();  
double[] rates = {36.25, 25.4, 18.34, 35.7,23.78};  
for (int i=0; i<rates.length; i++)  
    rateVector.add(rates[i]);
```

- 当从**Vector**中取出时，需要塑型成相应的包裹类型，之后再还原为原始类型。代码如下：

```
double sum = 0.0;  
for (int i=0; i<rateVector.size();i++)  
    sum += (rateVector.get(i));  
return sum;
```

集
合

7.3.3 Enumeration及Iterator接口

- 集合类对象中每个元素的遍历方法

```
for (int i=0; i<v.size();i++) {  
    Customer c = (Customer)v.get(i);  
    System.out.println(c.getName());  
}
```

集
合

- **Enumeration / Iterator**

- 能够从集合类对象中提取每一个元素，并提供了用于遍历元素的方法
- Java中的许多方法（如elements()）都返回Enumeration类型的对象，而不是返回集合类对象
- Enumeration接口不能用于ArrayList对象，而Iterator接口既可以用于ArrayList对象，也可以用于Vector对象

7.3.3 Enumeration及Iterator接口(续)

——Enumeration接口

- Enumeration接口（1.0版）

- 提供了两个实例方法

- hasMoreElements() —— 判断是否还有剩下的元素；
- nextElement() —— 取得下一个元素。

- 遍历集合类对象v中的每个元素可使用下面代码完成：

```
Enumeration e = v.elements();
while (e.hasMoreElements()) {
    Customer c = (Customer)v.nextElement();
    System.out.println(c.getName());
}
```

集
合

7.3.3 Enumeration及Iterator接口(续)

——例7_6

- Enumeration类使用举例

```
import java.util.Vector;
import java.util.Enumeration;
public class EnumerationTester {
    public static void main(String args[]) {
        Enumeration  days;
        Vector  dayNames = new Vector();
        dayNames.add("Sunday");
        dayNames.add("Monday");
        dayNames.add("Tuesday");
        dayNames.add("Wednesday");
        dayNames.add("Thursday");
        dayNames.add("Friday");
        dayNames.add("Saturday");
        days = dayNames.elements();
        while (days.hasMoreElements())
            System.out.println(days.nextElement());
    }
}
```

集
合

7.3.3 Enumeration及Iterator接口(续)

——例7_6运行结果

集合

- 运行结果

Sunday

Monday

Tuesday

Wednesday

Thursday

Friday

Saturday

- 常见编程错误

- 在一次循环中调用两次nextElement()方法
- 在遍历的过程中从集合对象中去除元素，具体见例7-7

7.3.3 Enumeration及Iterator接口(续)

——例7_7

- Enumeration类的问题举例

```
import java.util.Vector;
import java.util.Enumeration;
public class EnumerationTester2 {
    public static void main(String args[]) {
        String[] num = {"one", "two", "three", "four", "five",
                        "six", "seven", "eight", "nine", "ten"};
        Vector aVector = new Vector(java.util.Arrays.asList(num));
        System.out.println("Before Vector: " + aVector);
        Enumeration nums = aVector.elements();
        while(nums.hasMoreElements()) {
            String aString = (String)nums.nextElement();
            System.out.println(aString);
            if (aString.length() > 4) aVector.remove(aString);!!!!
        }
        System.out.println("After Vector: " + aVector);
    }
}
```

集合

7.3.3 Enumeration及Iterator接口(续)

——例7_7运行结果

集合

- 运行结果

Before Vector: [one, two, three, four, five, six, seven, eight, nine, ten]

one

two

three

five

six

seven

nine

ten

After Vector: [one, two, four, five, six, eight, nine, ten]

- 可见，长度大于4的字符串并没有如预期的那样都从向量中去除。这是**Enumeration**类存在的问题。因此，如果需要在遍历的过程中去除对象，就不要使用**Enumeration**类

7.3.3 Enumeration及Iterator接口(续)

——Iterator接口

- **Iterator接口（1.2版，替代Enumeration）**

- 也是一个遍历集合元素的工具，是对Enumeration接口的改进，因此在遍历集合元素时，优先选用Iterator接口
- 与Enumeration不同，具有从正在遍历的集合中去除对象的能力
- 具有如下三个实例方法，可见相对于Enumeration接口简化了方法名
 - **hasNext()** —— 判断是否还有元素
 - **next()** —— 取得下一个元素
 - **remove()** —— 去除一个元素。注意是从集合中去除最后调用next()返回的元素，而不是从Iterator类中去除

集
合

7.3.3 Enumeration及Iterator类(续)

——例7_8

- Iterator类使用举例

```
import java.util.Vector;
import java.util.Iterator;
public class IteratorTester {
    public static void main(String args[]) {
        String[] num = {"one", "two", "three", "four", "five",
                        "six", "seven", "eight", "nine", "ten"};
        Vector aVector = new Vector(java.util.Arrays.asList(num));
        System.out.println("Before Vector: " + aVector);
        Iterator nums = aVector.iterator();
        while(nums.hasNext()) {
            String aString = (String)nums.next();
            System.out.println(aString);
            if (aString.length() > 4)    nums.remove();
        }
        System.out.println("After Vector: " + aVector);
    }
}
```

集
合

7.3.3 Enumeration及Iterator类(续)

——例7_8运行结果

集合

- 运行结果

Before Vector: [one, two, three, four, five, six, seven, eight, nine, ten]

one

two

three

four

five

six

seven

eight

nine

ten

After Vector: [one, two, four, five, six, nine, ten]

- 在遍历的过程中，**Iterator**类对象能够与其对应的集合对象保持一致，没有元素被遗漏，因此能够得到正确的结果

7.3.4 Map接口及其类层次

- Map接口

- 以该接口为根的集合类

- 用于存储“关键字”(key)和“值”(value)的元素对，其中每个关键字映射到一个值
- 当需要通过关键字实现对值的快速存取时使用

- 声明的抽象方法主要有

- 查询方法
- 修改方法

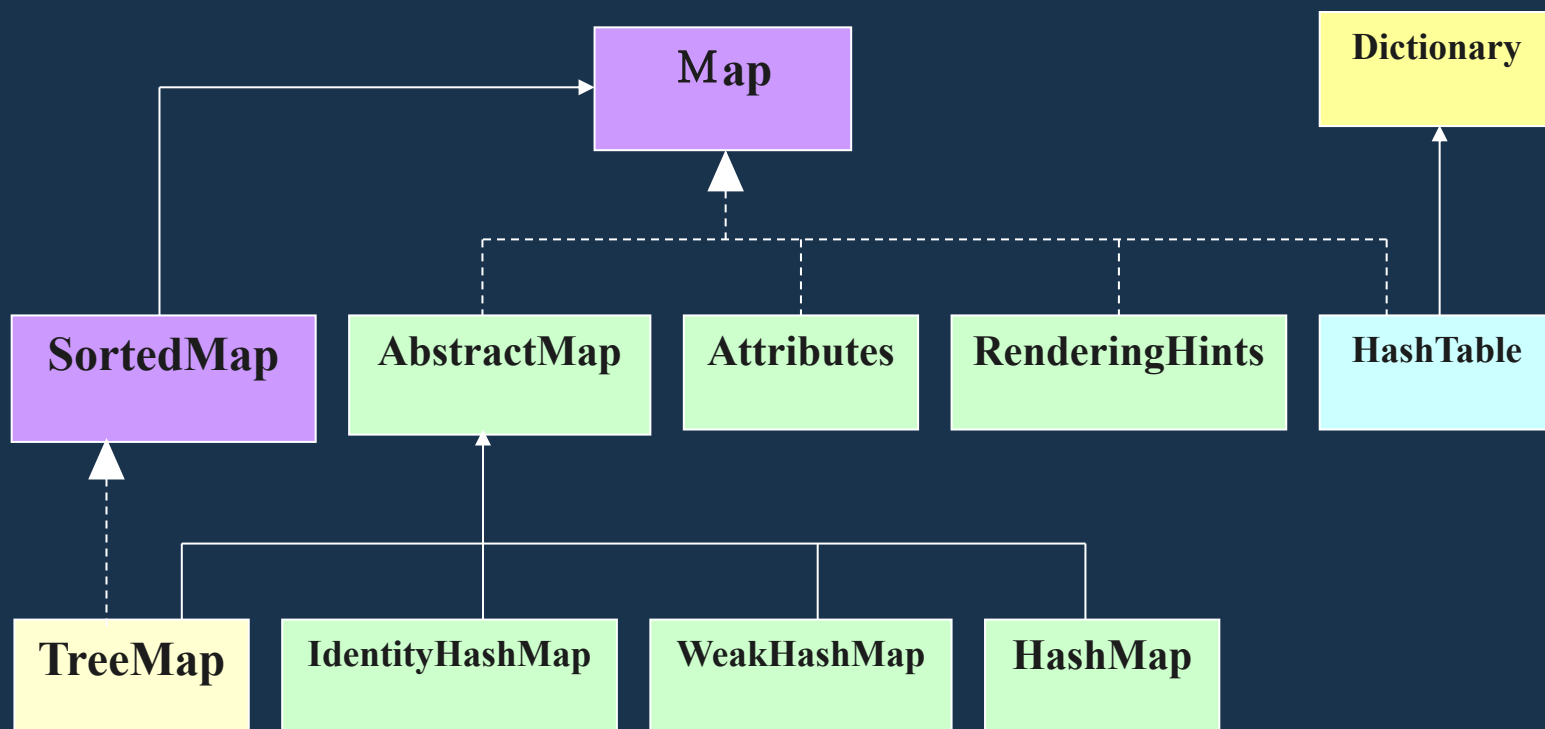
- 两个主要实现类

- Hashtable (1.0)
- HashMap (1.2)

集
合

7.3.4 Map接口及其类层次(续)

集合



7.3.4 Map接口及其类层次(续)

- 查询方法

- **int** `size()` —— 返回Map中的元素个数
- **boolean** `isEmpty()` —— 返回Map中是否包含元素，如不包括任何元素，则返回true
- **boolean** `containsKey(Object key)` —— 判断给定的参数是否是Map中的一个关键字(key)
- **boolean** `containsValue(Object val)` —— 判断给定的参数是否是Map中的一个值(value)
- **Object** `get(Object key)` —— 返回Map中与给定关键字相关联的值(value)
- **Collection** `values()` —— 返回包含Map中所有值(value)的Collection对象
- **Set** `keySet()` —— 返回包含Map中所有关键字(key)的Set对象
- **Set** `entrySet()` —— 返回包含Map中所有项的Set对象

集合

7.3.4 Map接口及其类层次(续)

- 修改方法

- `Object put(Object key, Object val)` —— 将给定的关键字(key)/值(value)对加入到Map对象中。其中关键字(key)必须唯一，否则，新加入的值会取代Map对象中已有的值
- `void putAll(Map m)` —— 将给定的参数Map中的所有项加入到接收者Map对象中
- `Object remove(Object key)` —— 将关键字为给定参数的项从Map对象中删除
- `void clear()` —— 从Map对象中删除所有的项

7.3.5 哈希表(HashTable, HashMap)

- 哈希表

- 也称为散列表，是用来存储群体对象的集合类结构，其两个常用的类是HashTable及HashMap
- 哈希表存储对象的方式与前面所讲的数组，Vector及ArrayList不同
 - 数组，Vector及ArrayList中对象的存储位置是随机的，即对象本身与其存储位置之间没有必然的联系。因此查找一个对象时，只能以某种顺序（如顺序查找，二分查找）与各个元素进行比较，如果数组或向量中的元素数量很庞大时，查找的效率必然降低
 - 哈希表中，对象的存储位置和对象的关键属性k之间有一个特定的对应关系f，我们称之为哈希(Hash)函数。它使每个对象与一个唯一的存储位置相对应。因而在查找时，只要根据待查对象的关键属性k，计算f(k)的值即可知其存储位置

7.3.5 哈希表(HashTable, HashMap)(续)

——哈希表相关的主要概念

集合

- 哈希表相关的一些主要概念
 - 容量（**capacity**）—— 哈希表的容量不是固定的，随对象的加入，其容量可以自动扩充
 - 关键字 / 键（**key**）—— 每个存储的对象都需要有一个关键字**key**，**key**可以是对象本身，也可以是对象的一部分（如对象的某一个属性）
 - 哈希码（**hash code**）—— 要将对象存储到**HashTable**，就需要将其关键字**key**映射到一个整型数据，称为**key**的哈希码（**hash code**）
 - 哈希函数（**hash function**）—— 返回对象的哈希码
 - 项（**item**）—— 哈希表中的每一项都有两个域：关键字域**key**及值域**value**（即存储的对象）。**key**及**value**都可以是任意的**Object**类型的对象，但不能为空(**null**)，**HashTable**中的所有关键字都是唯一的
 - 装填因子（**load factor**）—— (表中填入的项数) / (表的容量)

7.3.5 哈希表(HashTable, HashMap)(续)

——HashTable的常用方法1

- 构造方法
 - Hashtable(); // 初始容量为101, 最大装填因子为0.75
 - Hashtable(int capacity);
 - Hashtable(int capacity, float maxLoadFactor);
- **Object put(Object key, Object value)** —— 值value以key为其关键字加入到哈希表中, 如果此关键字在表中不存在, 则返回null, 否则返回表中存储的value

```
Hashtable aPhoneBook = new Hashtable();  
aPhoneBook.put("Zhang Lei", "010-84256712");  
aPhoneBook.put("Zhu Yongqin", "010-82957788");  
aPhoneBook.put("Liu Na", "010-80791234");  
System.out.println(aPhoneBook);
```

```
// 显示{ Liu Na=010-80791234, Zhu Yongqin=010-82957788, Zhang  
Lei=010-84256712 }
```

7.3.5 哈希表(HashTable, HashMap)(续)

——HashTable的常用方法2

集合

- **Object get(Object key)** —— 返回关键字为key的值value,如果不存在,则返回null。如

```
Hashtable aPhoneBook = new Hashtable();  
aPhoneBook.put("Zhang Lei", "010-84256712");  
aPhoneBook.put("Zhu Yongqin", "010-82957788");  
aPhoneBook.get("Zhang Lei");    // 返回"010-84256712"  
aPhoneBook.get("Zhu Yongqin");  // 返回"010-82957788"  
aPhoneBook.get("Liu Ling");     // 返回null
```

- **Object remove(Object key)** —— 将键 / 值对从表中去除,并返回从表中去除的值,如果不存在,则返回null。如

```
Hashtable aPhoneBook = new Hashtable();  
aPhoneBook.put("Zhu Yongqin", "010-82957788");  
aPhoneBook.put("Liu Na", "010-80791234");  
aPhoneBook.remove("Zhu Yongqin");  
aPhoneBook.remove("010-80791234"); // 不出错,但返回null  
System.out.println(aPhoneBook); // 显示{ Liu Na=010-80791234}
```

7.3.5 哈希表(HashTable, HashMap)(续)

——HashTable的常用方法3

集合

- **boolean isEmpty()** —— 判断哈希表是否为空
- **boolean containsKey(Object key)** —— 判断给定的关键字是否在哈希表中
- **boolean contains(Object value)** —— 测试此映射表中是否存在与指定值关联的键
- **boolean containsValue(Object value)** —— 如果Hashtable 将一个或多个键映射到此值，则返回 true
- **void clear()** —— 将哈希表清空
- **Enumeration elements()** —— 返回包含值的Enumeration对象
- **Enumeration keys()** —— 返回包含关键字的Enumeration对象
- **HashMap**类与**HashTable**类很相似，只是**HashTable**类不允许有空的关键字，而**HashMap**类允许

J2SE 5.0中的泛型

- 在以前的JDK的版本中，很多集合框架的函数接受一个**Object**参数。
 - **collections**是一个能持有任何对象的多用途工具，但带来了不良的后果。
- 泛型的好处
 - 编译时的严格类型检查。
 - 消除了绝大多数的类型转换。

J2SE 5.0中的泛型

- 例：使用 **ArrayList<E>**

```
ArrayList<Integer> aList = new ArrayList<Integer>();  
    aList.add(new Integer(1));  
    // ...  
    Integer myInteger = aList.get(0);
```


J2SE 5.0中的泛型

- 例：泛型类如何定义

```
public class TestGenerics<E>
{
    Collection<E> col;
    public void doSth(E elem)
    {
        col.add(elem);
        // ...
    }
}
```

J2SE 5.0中的泛型

- 例：泛型方法的定义

```
public static <T extends SomeClass>  
void add (Collection<T> c, T elem) {  
    c.add(elem);  
}
```

7.4 本章小结

- 本章内容

- 本章从对群体数据进行组织与管理的角度，介绍了对象数组、**Java**集合框架，以及常用的集合接口和类

- 本章要求

- 了解群体数据的组织以及**Java**的集合框架
- 理解数组、**Collection**、**Map**的区别，能够区分各自的使用范围以及优缺点
- 掌握**Arrays**类、**Vector**类、**ArrayList**类、**Enumeration**接口、**Iterator**接口、**HashTable**、**HashMap**类的常用方法